PRACTICAL EXERCISE
Google BiQuery
QUESTION 1

```sql
SELECT * FROM `practical-493418.Retail_sale.Data` LIMIT 1000;

-- Question 1 WHERE Clause
SELECT *
FROM `practical-493418.Retail_sale.Data`
WHERE Date BETWEEN '2023-01-31' AND '2023-12-31';



-- Question 2 Fltering + Conditions
SELECT *
```

Query completed

Query results

Chat | Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price pe
1	191	2023-10-18	CUST191	Male	64	Beauty	1	
2	204	2023-09-28	CUST204	Male	39	Beauty	1	
3	230	2023-04-23	CUST230	Male	54	Beauty	1	

QUESTION 2

```sql
-- Question 2 Fltering + Conditions
SELECT *
FROM `practical-493418.Retail_sale.Data`
WHERE `Total Amount` > (SELECT AVG (`Total Amount`)
FROM practical-493418.Retail_sale.Data);
```

Query completed

Using on-demand processing quota

Query results

Chat | Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per
1	21	2023-01-14	CUST021	Female	50	Beauty	1	
2	28	2023-04-23	CUST028	Female	43	Beauty	1	
3	128	2023-07-05	CUST128	Male	25	Beauty	1	

QUESTION 3

```sql
-- Question 3 Aggregate Functions - Calculating the total revenue (sum of Total Amount)
SELECT
SUM(`Total Amount`) AS total_revenue
FROM `practical-493418.Retail_sale.Data`;
```

Query completed

Using on-demand processing quota

Query results

Chat

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	total_revenue
1	456000

QUESTION 4

```sql
-- Question 4 DISTINCT
SELECT DISTINCT('Product Category')
FROM practical-493418.Retail_sale.Data;

--- Question 5 GROUP BY
SELECT
```

This script will process 340.96 KB when run.

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	f0_
1	Product Category

QUESTION 5

```sql
-- Question 5 GROUP BY
SELECT
    `Product Category`,
    SUM(`Quantity`) AS `total_quantity_sold`
    FROM `practical-493418.Retail_sale.Data`
    GROUP BY `Product Category`
    ORDER BY total_quantity_sold
    DESC;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Executi

Row	Product Category	total_quantity_sold
1	Clothing	894
2	Electronics	849
3	Beauty	771

QUESTION 6

```sql
--- Question 6 CASE Statement
SELECT
    `Customer ID`, Age,
  CASE
      WHEN Age < 30 THEN 'Youth'
      WHEN Age BETWEEN 30 and 59 THEN 'Adult'
      WHEN Age >= 60 THEN 'Senior'
      END AS Age_Group
      FROM `practical-493418.Retail_sale.Data`;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Customer ID	Age	Age_Group
1	CUST191	64	Senior
2	CUST204	39	Adult
3	CUST230	54	Adult
4	CUST232	43	Adult
5	CUST309	26	Youth
6	CUST310	28	Youth

QUESTION 7

```sql
--Question 7 Conditional Aggregation
SELECT Gender,
    COUNT(`Transaction ID`) AS TOTAL_QUANTITY
    FROM `practical-493418.Retail_sale.Data`
    WHERE `Total Amount` > 500
    GROUP BY Gender;
```

This script will process 340.96 KB when run.

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Gender	TOTAL_QUANTITY
1	Female	155
2	Male	144

QUESTION 8

```sql
-- Question 8 HAVING
SELECT `Product Category`,
    SUM(`Total Amount`) AS Total_Revenue
FROM `practical-493418.Retail_sale.Data`
GROUP BY `Product Category`
HAVING SUM(`Total Amount`) > 5000;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Product Category	Total_Revenue
1	Beauty	143515
2	Clothing	155580
3	Electronics	156905

QUESTION 9

```sql
-- Question 9 Calculated Fields
SELECT
    `Transaction ID`, `Price per unit`,
    CASE
        WHEN `Price per Unit` < 50 THEN 'Cheap'
        WHEN `Price per Unit` BETWEEN 50 AND 200 THEN 'Moderate'
        WHEN `pRICE PER uNIT` > 200 THEN 'Expensive'
END AS `Unit Cost Category`
FROM `practical-493418.Retail_sale.Data`;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Transaction ID	Price per unit	Unit Cost Category
1	191	25	Cheap
2	204	25	Cheap
3	230	25	Cheap
4	232	25	Cheap
5	309	25	Cheap
6	310	25	Cheap

QUESTION 10

```sql
--- Question 10 Combining WHERE + CASE
SELECT
  `Customer ID`, Age,
    `Total Amount`,
  CASE
    WHEN `Total Amount` > 1000 THEN 'High'
ELSE 'Low'
END AS Spending_Level
FROM `practical-493418.Retail_sale.Data`
WHERE Age >= 40;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Customer ID	Age	Total Amount	Spending_Level
1	CUST191	64	25	Low
2	CUST230	54	25	Low
3	CUST232	43	25	Low
4	CUST363	64	25	Low
5	CUST454	46	25	Low
6	CUST512	57	25	Low